Specialization

Concepts of Programming Languages

CAS CS 320

Outline

- Finish our demo of HM^-
- Discuss **specialization** and how it relates to principle types

Recap

Recall: Principal Types

$$\Gamma \vdash e : \tau \dashv \mathscr{C}$$

Recall: Principal Types

$$\Gamma \vdash e : \tau \dashv \mathscr{C}$$

$\mathscr{C}$ defines a *unification problem.* Given an MGU $\mathscr{S}$ we can get the "actual" type of e:

Recall: Principal Types

$$\Gamma \vdash e : \tau \dashv \mathscr{C}$$

$\mathscr{C}$ defines a *unification problem*. Given an MGU $\mathscr{S}$ we can get the "actual" type of e:

$\text{principal}(\tau, \mathscr{C}) = \forall \alpha_1 \ldots \forall \alpha_k . \mathscr{S}\tau$ where $\text{FV}(\mathscr{S}\tau) = \{\alpha_1, \ldots, \alpha_k\}$

Recall: Putting everything together

Recall: Putting everything together

<u>input:</u> program P (sequence of top-level let-expressions)

Recall: Putting everything together

<u>input:</u> program P (sequence of top-level let-expressions)

$\Gamma \leftarrow \varnothing$

Recall: Putting everything together

<u>input:</u> program P (sequence of top-level let-expressions)

$\Gamma \leftarrow \varnothing$

FOR EACH top-level let-expression let $x = e$ in P:

Recall: Putting everything together

<u>input:</u> program P (sequence of top-level let-expressions)

$\Gamma \leftarrow \varnothing$

FOR EACH top-level let-expression $\mathsf{let}\ x = e$ in P:

1. *Constraint-based inference:* Determine τ and $\mathscr{C}$ such that $\Gamma \vdash e : \tau \dashv \mathscr{C}$ is derivable

Recall: Putting everything together

<u>input:</u> program P (sequence of top-level let-expressions)

$\Gamma \leftarrow \varnothing$

FOR EACH top-level let-expression $\text{let } x = e$ in P:

1. *Constraint-based inference:* Determine τ and $\mathscr{C}$ such that $\Gamma \vdash e : \tau \dashv \mathscr{C}$ is derivable

2. *Unification:* Solve $\mathscr{C}$ to get a most general unifier $\mathscr{S}$ (**TYPE ERROR** if this fails)

Recall: Putting everything together

<u>input:</u> program P (sequence of top-level let-expressions)

$\Gamma \leftarrow \varnothing$

FOR EACH top-level let-expression $\text{let } x = e$ in P:

1. *Constraint-based inference:* Determine τ and $\mathscr{C}$ such that $\Gamma \vdash e : \tau \dashv \mathscr{C}$ is derivable

2. *Unification:* Solve $\mathscr{C}$ to get a most general unifier $\mathscr{S}$ (**TYPE ERROR** if this fails)

3. *Generalization:* Quantify over the free variables in $\mathscr{S}\tau$ to get the principle type $\forall \alpha_1 \ldots \forall \alpha_k . \mathscr{S}\tau$ of e

Recall: Putting everything together

<u>input:</u> program P (sequence of top-level let-expressions)

$\Gamma \leftarrow \varnothing$

FOR EACH top-level let-expression $\text{let } x = e$ in P:

1. *Constraint-based inference:* Determine τ and $\mathscr{C}$ such that $\Gamma \vdash e : \tau \dashv \mathscr{C}$ is derivable
2. *Unification:* Solve $\mathscr{C}$ to get a most general unifier $\mathscr{S}$ (**TYPE ERROR** if this fails)
3. *Generalization:* Quantify over the free variables in $\mathscr{S}\tau$ to get the principle type $\forall \alpha_1 \ldots \forall \alpha_k . \mathscr{S}\tau$ of e
4. Add $(x : \forall \alpha_1 \ldots \forall \alpha_k . \mathscr{S}\tau)$ to Γ

Recall: As a Type System

$$\frac{}{\Gamma \vdash \epsilon}\ \texttt{(emptyProg)}$$

$$\frac{\Gamma \vdash e : \tau \dashv \mathscr{C} \qquad \Gamma, x : \mathsf{principal}(\tau, \mathscr{C}) \vdash P}{\Gamma \vdash \mathsf{let}\ x = e\ P}\ \texttt{(topLet)}$$

We can also express this as a type system with judgments of the form $\Gamma \vdash P$, where P is a program (note there is no ":")

demo

(HM⁻)

Recall: HM⁻ (Typing Applications)

$$\frac{\Gamma \vdash e_1 : \tau_1 \dashv \mathscr{C}_1 \qquad \Gamma \vdash e_2 : \tau_2 \dashv \mathscr{C}_2 \qquad \alpha \texttt{ is fresh}}{\Gamma \vdash e_1 e_2 : \alpha \dashv \tau_1 \doteq \tau_2 \to \alpha, \mathscr{C}_1, \mathscr{C}_2} \ \texttt{(app)}$$

Recall: HM⁻ (Typing Variables)

$$\frac{(x : \forall \alpha_1 . \forall \alpha_2 \ldots \forall \alpha_k . \tau) \in \Gamma \qquad \beta_1, \ldots, \beta_k \ \texttt{are fresh}}{\Gamma \vdash x : [\beta_1/\alpha_1] \ldots [\beta_k/\alpha_k]\tau \dashv \varnothing} \ \texttt{(var)}$$

Specialization

Recall: HM$^-$ (Syntax)

$$
\begin{aligned}
e ::= \; & \lambda x \,.\, e \mid ee \\
& \mid \text{let } x = e \text{ in } e \\
& \mid \text{if } e \text{ then } e \text{ else } e \\
& \mid e + e \mid e = e \\
& \mid n \mid x \\
\sigma ::= \; & \text{int} \mid \text{bool} \mid \alpha \mid \sigma \to \sigma \\
\tau ::= \; & \sigma \mid \forall \alpha \,.\, \tau
\end{aligned}
$$

Recall: HM$^-$ (Typing)

$$\frac{n \texttt{ is an integer}}{\Gamma \vdash n : \mathsf{int} \dashv \varnothing} \ \texttt{(int)}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \dashv \mathscr{C}_1 \qquad \Gamma \vdash e_2 : \tau_2 \dashv \mathscr{C}_2 \qquad \Gamma \vdash e_3 : \tau_3 \dashv \mathscr{C}_3}{\Gamma \vdash \mathsf{if}\ e_1\ \mathsf{then}\ e_2\ \mathsf{else}\ e_3 : \tau_3 \dashv \tau_1 \doteq \mathsf{bool}, \tau_2 \doteq \tau_3, \mathscr{C}_1, \mathscr{C}_2, \mathscr{C}_3} \ \texttt{(if)}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \dashv \mathscr{C}_1 \qquad \Gamma \vdash e_2 : \tau_2 \dashv \mathscr{C}_2}{\Gamma \vdash e_1 = e_2 : \mathsf{bool} \dashv \tau_1 \doteq \tau_2, \mathscr{C}_1, \mathscr{C}_2} \ \texttt{(eq)}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \dashv \mathscr{C}_1 \qquad \Gamma \vdash e_2 : \tau_2 \dashv \mathscr{C}_2}{\Gamma \vdash e_1 + e_2 : \mathsf{int} \dashv \tau_1 \doteq \mathsf{int}, \tau_2 \doteq \mathsf{int}, \mathscr{C}_1, \mathscr{C}_2} \ \texttt{(add)}$$

$$\frac{\alpha \texttt{ is fresh} \qquad \Gamma, x : \alpha \vdash e : \tau \dashv \mathscr{C}}{\Gamma \vdash \lambda x . e : \alpha \to \tau \dashv \mathscr{C}} \ \texttt{(fun)}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \dashv \mathscr{C}_1 \qquad \Gamma \vdash e_2 : \tau_2 \dashv \mathscr{C}_2 \qquad \alpha \texttt{ is fresh}}{\Gamma \vdash e_1 e_2 : \alpha \dashv \tau_1 \doteq \tau_2 \to \alpha, \mathscr{C}_1, \mathscr{C}_2} \ \texttt{(app)}$$

HM$^-$ (Typing Let-Expressions)

$$\frac{\Gamma \vdash e_1 : \tau_1 \dashv \mathscr{C}_1 \qquad \Gamma, x : \tau_1 \vdash e_2 : \tau_2 \dashv \mathscr{C}_2}{\Gamma \vdash \mathsf{let}\ x = e_1\ \mathsf{in}\ e_2 : \tau_2 \dashv \mathscr{C}_1, \mathscr{C}_2} \ \texttt{(let)}$$

Note that this rule doesn't allow let-functions to be polymorphic

Example

Recall: HM⁻ (Typing Variables)

$$\frac{(x : \forall \alpha_1 . \forall \alpha_2 \ldots \forall \alpha_k . \tau) \in \Gamma \qquad \beta_1, \ldots, \beta_k \text{ are fresh}}{\Gamma \vdash x : [\beta_1/\alpha_1] \ldots [\beta_k/\alpha_k]\tau \dashv \varnothing} \text{ (var)}$$

If x is declared in Γ, then x can be given the type τ *with all free variables replaced by* ***fresh variables***

This is where the polymorphism magic happens

Fresh variables can be unified with anything

An Alternative Formulation

$$\Gamma \vdash e : \tau$$

It's possible to give a type system for HM⁻ *without* constraints

It's very similar to our 320Caml system, but with some rules for dealing with **quantification** and **specialization**

HM$^-$ (Alternative Typing)

$$\frac{n \texttt{ is an integer}}{\Gamma \vdash n : \mathsf{int}} \ (\texttt{int})$$

$$\frac{\Gamma \vdash e_1 : \mathsf{bool} \qquad \Gamma \vdash e_2 : \tau \qquad \Gamma \vdash e_3 : \tau}{\Gamma \vdash \mathsf{if}\ e_1\ \mathsf{then}\ e_2\ \mathsf{else}\ e_3 : \tau} \ (\texttt{if})$$

$$\frac{\Gamma \vdash e_1 : \tau \qquad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1 = e_2 : \mathsf{bool}} \ (\texttt{eq})$$

$$\frac{\Gamma \vdash e_1 : \mathsf{int} \qquad \Gamma \vdash e_2 : \mathsf{int}}{\Gamma \vdash e_1 + e_2 : \mathsf{int}} \ (\texttt{add})$$

$$\frac{\Gamma, x : \tau_1 \vdash e : \tau_2 \dashv \mathscr{C}}{\Gamma \vdash \lambda x\,.\,e : \tau_1 \to \tau_2 \dashv \mathscr{C}} \ (\texttt{fun})$$

$$\frac{\Gamma \vdash e_1 : \tau_2 \to \tau \qquad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash e_1 e_2 : \tau} \ (\texttt{app})$$

$$\frac{\tau_1 \texttt{ is a monotype} \qquad \Gamma \vdash e_1 : \tau_1 \qquad \Gamma, x : \tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash \mathsf{let}\ x = e_1\ \mathsf{in}\ e_2 : \tau_2} \ (\texttt{let})$$

HM$^-$ (Alternative Typing)

familiar rules

$$\frac{n \texttt{ is an integer}}{\Gamma \vdash n : \mathsf{int}} \ (\texttt{int})$$

$$\frac{\Gamma \vdash e_1 : \mathsf{bool} \qquad \Gamma \vdash e_2 : \tau \qquad \Gamma \vdash e_3 : \tau}{\Gamma \vdash \mathsf{if}\ e_1\ \mathsf{then}\ e_2\ \mathsf{else}\ e_3 : \tau} \ (\texttt{if})$$

$$\frac{\Gamma \vdash e_1 : \tau \qquad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1 = e_2 : \mathsf{bool}} \ (\texttt{eq})$$

$$\frac{\Gamma \vdash e_1 : \mathsf{int} \qquad \Gamma \vdash e_2 : \mathsf{int}}{\Gamma \vdash e_1 + e_2 : \mathsf{int}} \ (\texttt{add})$$

$$\frac{\Gamma, x : \tau_1 \vdash e : \tau_2 \dashv \mathscr{C}}{\Gamma \vdash \lambda x . e : \tau_1 \to \tau_2 \dashv \mathscr{C}} \ (\texttt{fun})$$

$$\frac{\Gamma \vdash e_1 : \tau_2 \to \tau \qquad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash e_1 e_2 : \tau} \ (\texttt{app})$$

$$\frac{\tau_1 \texttt{ is a monotype} \qquad \Gamma \vdash e_1 : \tau_1 \qquad \Gamma, x : \tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash \mathsf{let}\ x = e_1\ \mathsf{in}\ e_2 : \tau_2} \ (\texttt{let})$$

Generalization and Specialization

$$\frac{\Gamma \vdash e : \tau \qquad \alpha \texttt{ not free in } \Gamma}{\Gamma \vdash e : \forall \alpha . \tau} \ \texttt{(gen)} \qquad \frac{(x : \tau) \in \Gamma \qquad \tau \sqsubseteq \tau'}{\Gamma \vdash x : \tau'} \ \texttt{(var)}$$

Generalization and Specialization

$$\frac{\Gamma \vdash e : \tau \qquad \alpha \texttt{ not free in } \Gamma}{\Gamma \vdash e : \forall \alpha . \tau} \ \texttt{(gen)} \qquad \frac{(x : \tau) \in \Gamma \qquad \tau \sqsubseteq \tau'}{\Gamma \vdash x : \tau'} \ \texttt{(var)}$$

The main difference: we introduce a notion of **specialization** which allows us to *instantiate* polymorphic functions at particular types

Generalization and Specialization

$$\frac{\Gamma \vdash e : \tau \qquad \alpha \text{ not free in } \Gamma}{\Gamma \vdash e : \forall \alpha . \tau} \text{ (gen)} \qquad \frac{(x : \tau) \in \Gamma \qquad \tau \sqsubseteq \tau'}{\Gamma \vdash x : \tau'} \text{ (var)}$$

The main difference: we introduce a notion of **specialization** which allows us to *instantiate* polymorphic functions at particular types

"$\sqsubseteq$" defined a *partial order* on type schemes

Example

Specialization (Informal)

$$\forall \alpha_1 \ldots \forall \alpha_m . \tau \sqsubseteq \forall \beta_1 \ldots \forall \beta_n . \tau'$$

A type scheme T_2 **specializes** T_1**,** written $T_1 \sqsubseteq T_2$ if T_2 the result of

1. instantiating the bound variables of T_1
2. generalizing over variables introduced by the instantiation

Specialization (Formal)

$$\frac{\begin{array}{c}\tau_1, \ldots, \tau_m \text{ are monotypes} \\ \tau' = [\tau_m/\alpha_m]\ldots[\tau_1/\alpha_1]\tau \\ \beta_1, \ldots, \beta_n \notin \mathsf{FV}(\tau) \backslash \{\alpha_1, \ldots, \alpha_m\}\end{array}}{\forall \alpha_1 \ldots \forall \alpha_m . \tau \sqsubseteq \forall \beta_1 \ldots \forall \beta_n . \tau'}$$

A *specialization* of a type scheme is an instantiation of its bound variable, together with some generalizations over remaining free variables

Examples

Examples

$$\forall \alpha \,.\, \forall \beta \,.\, \alpha \to \beta \to \alpha \sqsubseteq \forall \eta \,.\, \eta \to \mathsf{bool} \to \eta$$

$$\sqsubseteq \mathsf{int} \to \mathsf{bool} \to \mathsf{int}$$

Examples

$$\forall \alpha \,.\, \forall \beta \,.\, \alpha \to \beta \to \alpha \sqsubseteq \forall \eta \,.\, \eta \to \mathsf{bool} \to \eta$$
$$\sqsubseteq \mathsf{int} \to \mathsf{bool} \to \mathsf{int}$$

$$\forall \alpha \,.\, \forall \beta \,.\, \alpha \to \beta \to \alpha \sqsubseteq \forall \gamma \,.\, \mathsf{bool} \to (\gamma \to \gamma) \to \mathsf{bool}$$
$$\sqsubseteq \mathsf{bool} \to (\mathsf{int} \to \mathsf{int}) \to \mathsf{bool}$$

Examples

$$\forall \alpha . \forall \beta . \alpha \to \beta \to \alpha \sqsubseteq \forall \eta . \eta \to \mathsf{bool} \to \eta$$
$$\sqsubseteq \mathsf{int} \to \mathsf{bool} \to \mathsf{int}$$

$$\forall \alpha . \forall \beta . \alpha \to \beta \to \alpha \sqsubseteq \forall \gamma . \mathsf{bool} \to (\gamma \to \gamma) \to \mathsf{bool}$$
$$\sqsubseteq \mathsf{bool} \to (\mathsf{int} \to \mathsf{int}) \to \mathsf{bool}$$

$$\forall \alpha . \forall \beta . \alpha \to \beta \to \alpha \sqsubseteq \mathsf{bool} \to (\gamma \to \gamma) \to \mathsf{bool}$$
$$\not\sqsubseteq \mathsf{bool} \to (\mathsf{int} \to \mathsf{int}) \to \mathsf{bool}$$

Specialization and Principle Types

Specialization and Principle Types

Thm. If $\Gamma \vdash e : \tau'$ then there is a type τ and constraints $\mathscr{C}$ such that $\Gamma \vdash e : \tau \dashv \mathscr{C}$ and $\mathsf{principal}(\tau, \mathscr{C}) \sqsubseteq \tau'$

Specialization and Principle Types

Thm. If $\Gamma \vdash e : \tau'$ then there is a type τ and constraints $\mathscr{C}$ such that $\Gamma \vdash e : \tau \dashv \mathscr{C}$ and $\mathsf{principal}(\tau, \mathscr{C}) \sqsubseteq \tau'$

Thm. If $\Gamma \vdash e : \tau \dashv \mathscr{C}$ and $\mathsf{principal}(\tau, \mathscr{C}) \sqsubseteq \tau'$ then $\Gamma \vdash e : \tau'$

Specialization and Principle Types

Thm. If $\Gamma \vdash e : \tau'$ then there is a type τ and constraints $\mathscr{C}$ such that $\Gamma \vdash e : \tau \dashv \mathscr{C}$ and $\mathsf{principal}(\tau, \mathscr{C}) \sqsubseteq \tau'$

Thm. If $\Gamma \vdash e : \tau \dashv \mathscr{C}$ and $\mathsf{principal}(\tau, \mathscr{C}) \sqsubseteq \tau'$ then $\Gamma \vdash e : \tau'$

The principle type is the most general "lowest" type with respect to specialization

Example

$$\{f : \forall \alpha \,.\, \alpha \to \alpha\} \vdash f\ (f\ 2 = 2) : \mathsf{bool}$$

Why use constraints at all?

$$\frac{(x : \tau) \in \Gamma \qquad \tau \sqsubseteq \tau'}{\Gamma \vdash x : \tau'} \ \texttt{(var)} \qquad \frac{(x : \forall \alpha_1 . \forall \alpha_2 \ldots \forall \alpha_k . \tau) \in \Gamma \qquad \beta_1, \ldots, \beta_k \ \texttt{are fresh}}{\Gamma \vdash x : [\beta_1/\alpha_1] \ldots [\beta_k/\alpha_k]\tau \dashv \varnothing} \ \texttt{(var)}$$

Why use constraints at all?

$$\frac{(x : \tau) \in \Gamma \qquad \tau \sqsubseteq \tau'}{\Gamma \vdash x : \tau'} \ \texttt{(var)} \qquad \frac{(x : \forall \alpha_1 . \forall \alpha_2 \ldots \forall \alpha_k . \tau) \in \Gamma \qquad \beta_1, \ldots, \beta_k \ \texttt{are fresh}}{\Gamma \vdash x : [\beta_1/\alpha_1] \ldots [\beta_k/\alpha_k]\tau \dashv \varnothing} \ \texttt{(var)}$$

The alternative type rules are theoretically nice but not *algorithmic*

Why use constraints at all?

$$\frac{(x : \tau) \in \Gamma \qquad \tau \sqsubseteq \tau'}{\Gamma \vdash x : \tau'} \ \texttt{(var)} \qquad \frac{(x : \forall \alpha_1 . \forall \alpha_2 \ldots \forall \alpha_k . \tau) \in \Gamma \qquad \beta_1, \ldots, \beta_k \ \texttt{are fresh}}{\Gamma \vdash x : [\beta_1/\alpha_1] \ldots [\beta_k/\alpha_k]\tau \dashv \varnothing} \ \texttt{(var)}$$

The alternative type rules are theoretically nice but not *algorithmic*

How do I choose which specialization to use in a derivation?

Why use constraints at all?

$$\frac{(x : \tau) \in \Gamma \qquad \tau \sqsubseteq \tau'}{\Gamma \vdash x : \tau'} \ \texttt{(var)} \qquad \frac{(x : \forall \alpha_1 . \forall \alpha_2 \ldots \forall \alpha_k . \tau) \in \Gamma \qquad \beta_1, \ldots, \beta_k \ \texttt{are fresh}}{\Gamma \vdash x : [\beta_1/\alpha_1] \ldots [\beta_k/\alpha_k]\tau \dashv \varnothing} \ \texttt{(var)}$$

The alternative type rules are theoretically nice but not *algorithmic*

How do I choose which specialization to use in a derivation?

Constraints allow us to determine *which* specializations we should use *after the fact*

Summary

The **principle type** of an expression is the most general type we could give it

Specialization defines a partial ordering on type schemes from most to least general

Our unification algorithm gives us a most general unifier, which we use to define the principle type of an expression